

Lecture 17: Control Flow Instructions

CMPS 221 – Computer Organization and Design

Control Flow Instructions

- **Branch** to some instruction if a condition is true, otherwise continue sequentially
 - `beq rs, rt, L1`
 - if `rs` equals `rt`, branch to the instruction labeled `L1`
 - `bne rs, rt, L1`
 - if `rs` $\neq$ `rt`, branch to the instruction labeled `L1`
- **Jump** to some instruction unconditionally
 - `j L1`
 - jump to the instruction labeled `L1`
 - `jr rs`
 - jump to the instruction at the address stored in `rs`
 - `jal L1` (discussed later)

Compiling If Statements

- C code:

```
if (i == j)
    f = g + h;
else
    f = g - h;
```

- f : \$s0, g : \$s1, h : \$s2, i : \$s3, j : \$s4

- Compiled MIPS code:

```
bne $s3, $s4, Else
add $s0, $s1, $s2
j    EndIf
```

```
Else: sub $s0, $s1, $s2
EndIf: ...
```

Assembler calculates addresses

More Conditional Operations

- What if the condition was a comparison, not equality?
 - Combine beq/bne with set-on-less-than
- Set on less than instructions
 - `slt rd, rs, rt`
 - if $rs < rt$, set `rd` to 1, otherwise set `rd` to 0
 - `slti rt, rs, constant`
 - if $rs < \text{constant}$, set `rt` to 1, otherwise set `rt` to 0

Compiling If Statements

- C code:

```
if (i < j)
    f = g + h;
else
    f = g - h;
```

- f : \$s0, g : \$s1, h : \$s2, i : \$s3, j : \$s4

- Compiled MIPS code:

```
slt $t0, $s3, $s4
beq $t0, $zero, Else
add $s0, $s1, $s2
j    EndIf
```

```
Else:  sub $s0, $s1, $s2
```

```
EndIf:  ...
```

Branch Instruction Design

- Why not b1t, bge, etc?
- Hardware for $<$, $\geq$, ... slower than $=$, $\neq$
 - Combining with branch involves more work per instruction, requiring a slower clock
 - All instructions penalized!
- beq and bne are the common case
 - This is a good design compromise

Compiling Loop Statements

- C code:

```
while (i < j)
    i += 1;
```

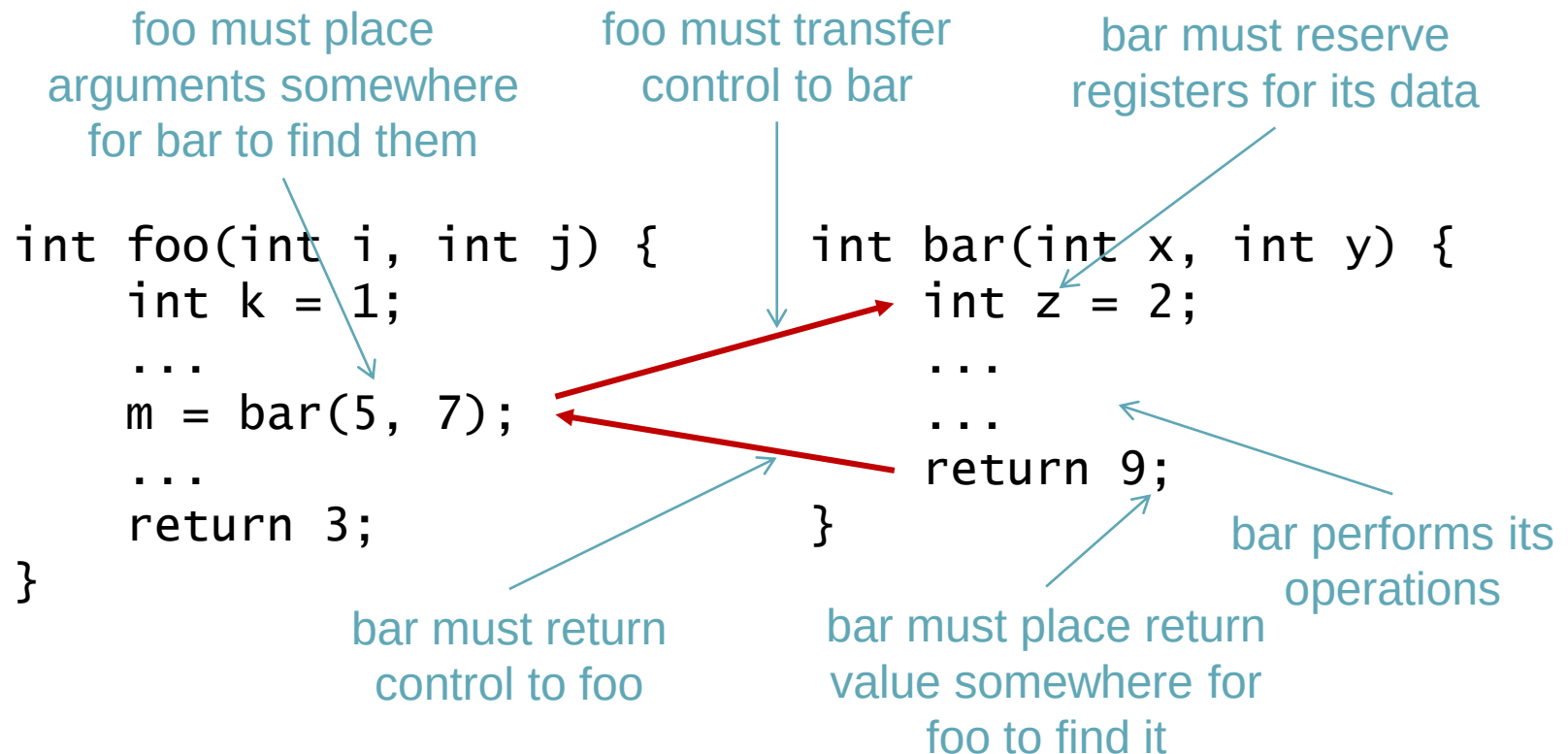
- $i : \$s3, j : \$s4$

- Compiled MIPS code:

```
Loop: slt $t0, $s3, $s4
      beq $t0, $zero, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: ...
```

Procedure Calling

- What needs to be done when a procedure calls another procedure?



Procedure Calling

- Steps required for a **caller** procedure to call a **callee** procedure
 1. Caller places arguments in registers for callee to use
 2. Caller transfers control to callee
 3. Callee acquires storage for its data
 4. Callee performs its operations
 5. Callee places result in register for caller
 6. Callee returns control to caller

More Register Names

- \$a0 – \$a3: arguments (reg's 4 – 7)
- \$v0, \$v1: result values (reg's 2 and 3)
- \$t0 – \$t9: temporaries (reg's 8-15, 24-25)
- \$s0 – \$s7: saved registers (reg's 16-23)
- \$gp: global pointer for static data (reg 28)
- \$sp: stack pointer (reg 29)
- \$fp: frame pointer (reg 30)
- \$ra: return address (reg 31)

Procedure Calling

- Steps required for a caller procedure to call a callee procedure
 1. Caller places arguments in registers for callee to use
 2. Caller transfers control to callee
 3. Callee acquires storage for its data
 4. Callee performs its operations
 5. Callee places result in register for caller
 6. Callee returns control to caller

Passing Arguments to Callee

- Caller stores arguments in reserved argument registers where callee reads them
 - \$a0 – \$a3: arguments (reg's 4 – 7)
- But the caller was using these registers for its own arguments
 - Caller needs to back up its own arguments somewhere else in the meantime (where discussed later)

Procedure Calling

- Steps required for a caller procedure to call a callee procedure
 1. Caller places arguments in registers for callee to use
 - 2. Caller transfers control to callee**
 3. Callee acquires storage for its data
 4. Callee performs its operations
 5. Callee places result in register for caller
 6. Callee returns control to caller

Transfer Control to Callee

- Caller executes jump and link instruction
 `jal calleeProcedureLabel`
 - Jumps to the callee's address
 - Puts the address of the caller's next instruction in a reserved register so the callee knows where to return
 - `$ra`: return address (reg 31)
- But the caller was using `$ra` for its own caller's address
 - Caller needs to back up its own `$ra` somewhere else in the meantime (where discussed later)

Procedure Calling

- Steps required for a caller procedure to call a callee procedure
 1. Caller places arguments in registers for callee to use
 2. Caller transfers control to callee
 3. **Callee acquires storage for its data**
 4. Callee performs its operations
 5. Callee places result in register for caller
 6. Callee returns control to caller

Acquiring Storage

- Recall: procedures store data in registers
 - \$t0 – \$t9: temporaries (reg's 8-15, 24-25)
 - \$s0 – \$s7: saved registers (reg's 16-23)
- Callee can store its data in these registers
- But the caller may have already been using these for its own data
 - Needs to back up the caller's data somewhere else in the meantime (where discussed later)
 - Who backs up the caller's data? The caller or callee?

Acquiring Storage

- Let's assume the caller backs up all the data
 - The caller does not know how many registers the callee needs
 - If the caller was using 10 registers and the callee only needs 4, the caller would have backed up 6 registers unnecessarily
- Let's assume the callee backs up all the data
 - The callee does not know how many registers the caller was using
 - If the caller was using 4 registers and the callee needs 10, the callee would need to back up 6 registers unnecessarily
- Compromise: split the work between the caller and callee

Acquiring Storage

- Split the work between the caller and callee
 - If caller uses any `$t_` registers, it must back them up before calling and restore them when callee returns
 - Where discussed later
 - If callee uses any `$s_` registers, it must back them up before using them and restore them before returning
 - Where discussed later

Procedure Calling

- Steps required for a caller procedure to call a callee procedure
 1. Caller places arguments in registers for callee to use
 2. Caller transfers control to callee
 3. Callee acquires storage for its data
 4. **Callee performs its operations**
 5. Callee places result in register for caller
 6. Callee returns control to caller

Procedure Calling

- Steps required for a caller procedure to call a callee procedure
 1. Caller places arguments in registers for callee to use
 2. Caller transfers control to callee
 3. Callee acquires storage for its data
 4. Callee performs its operations
 5. Callee places result in register for caller
 6. Callee returns control to caller

Returning Result to Caller

- Callee stores result in reserved return value registers where caller reads them
 - \$v0, \$v1: result values (reg's 2 and 3)

Procedure Calling

- Steps required for a caller procedure to call a callee procedure
 1. Caller places arguments in registers for callee to use
 2. Caller transfers control to callee
 3. Callee acquires storage for its data
 4. Callee performs its operations
 5. Callee places result in register for caller
 6. Callee returns control to caller

Return Control to Caller

- Callee jump to return address in register
 `jr $ra`
 - Jumps to the address stored in \$ra

Textbook Sections

- The content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - 2.7, 2.8